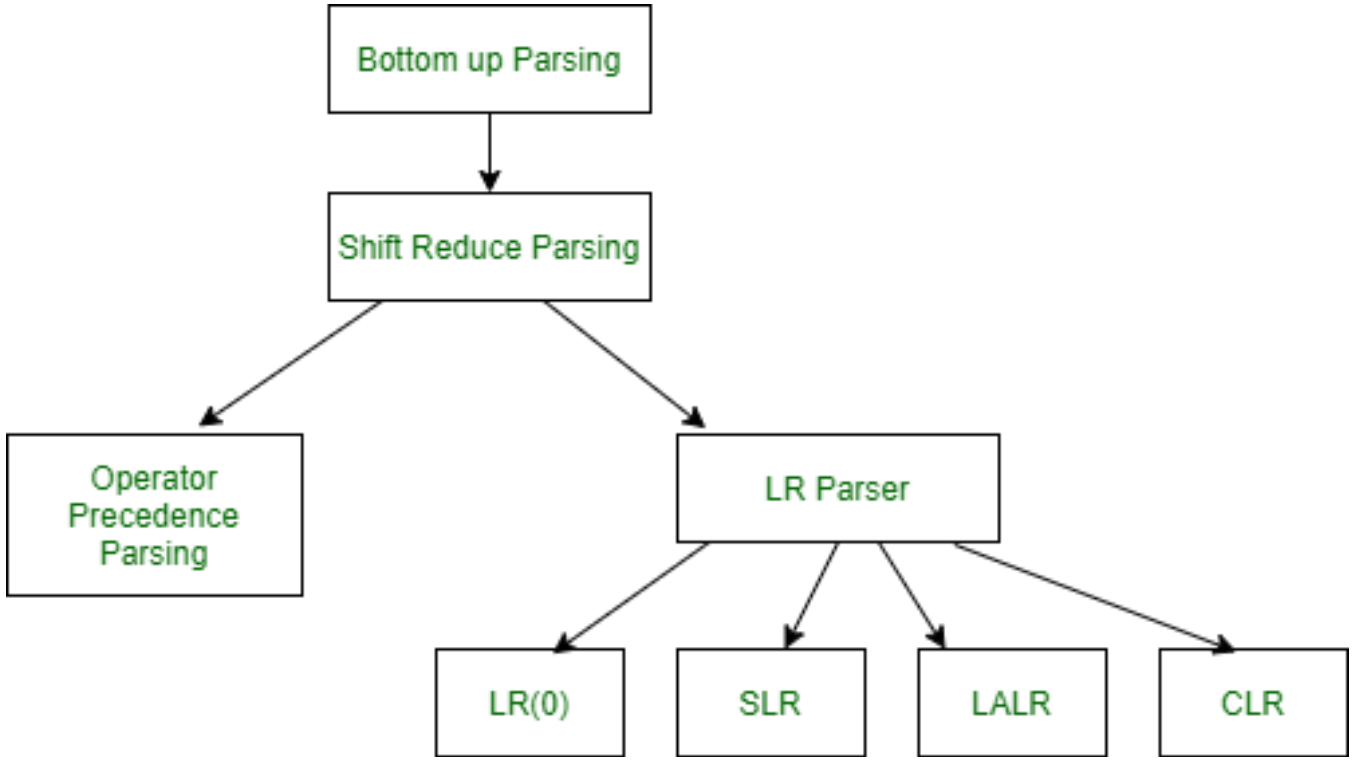


DS601PC: AUTOMATA THEORY AND COMPILER DESIGN

Lecture-14: LR Parser and UNIT V

Dr. Anjeneya Swami Kare
Assistant Professor
School of Computer and Information Sciences
askcs@uohyd.ac.in

Parsing Techniques



LR Parser

- LR parser is a bottom-up parser for context-free grammar
- Left-to-right scanning, Rightmost derivation (in reverse)
- LR parser reads their input from left to right and produces a right-most derivation
- A rightmost derivation is a type of derivation in which at each step, the rightmost non-terminal symbol is replaced (expanded) using production rules.

LR Parser

- LR(k) Parser: Scans the input Left-to-right (L) and constructs a Rightmost derivation in reverse (R) using k lookahead symbols.
- LR(0) → No lookahead
- LR(1) → Uses 1 symbol lookahead (most common)

LR(0) Parser

- Steps for constructing the LR parsing table:
  - Generate augmented grammar
  - LR(0) collection of items to be found
  - Defining 2 functions: goto(list of non-terminals) and action(list of terminals) in the parsing table.
- Example:
  - E->T+E| T
  - T->id

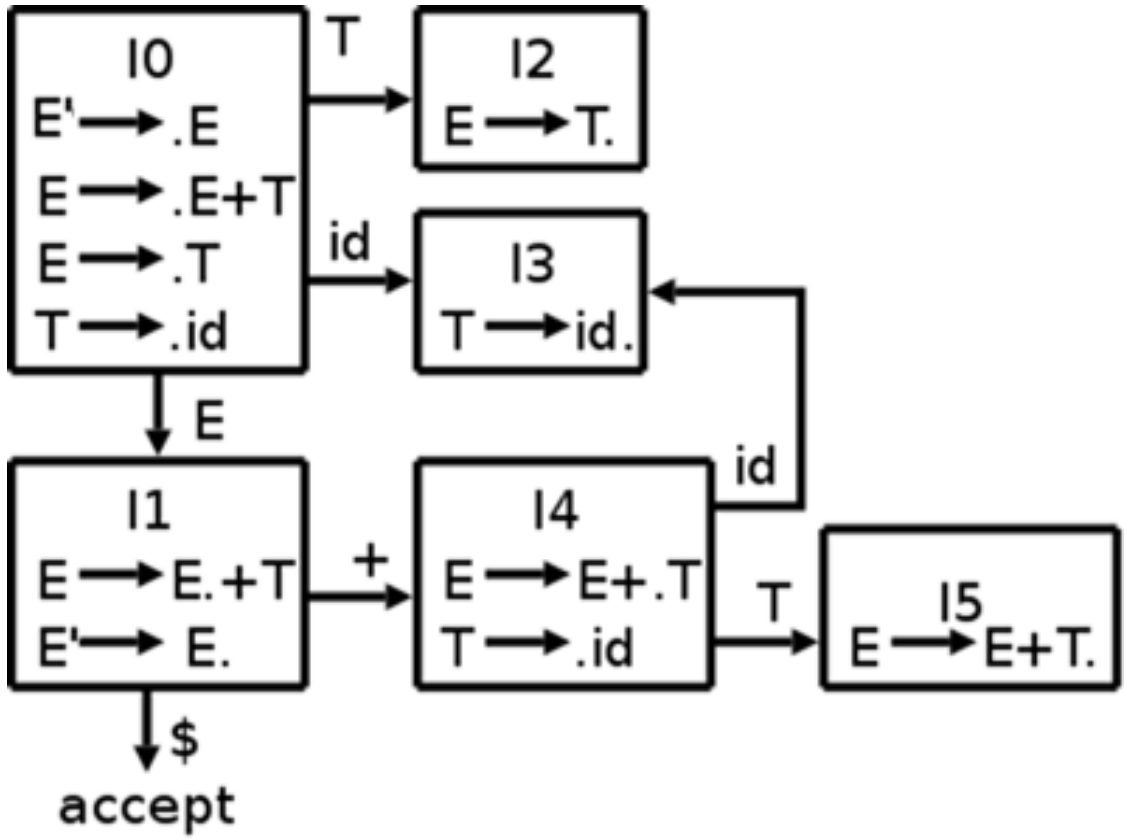


LR(0) Parser - Example

- Example:
  - E->T+E| T
  - T->id
- Augmented Grammar
  - E’->.E
  - E->.T+E
  - E->.T
  - T->.id

LR(0) Parser - Example

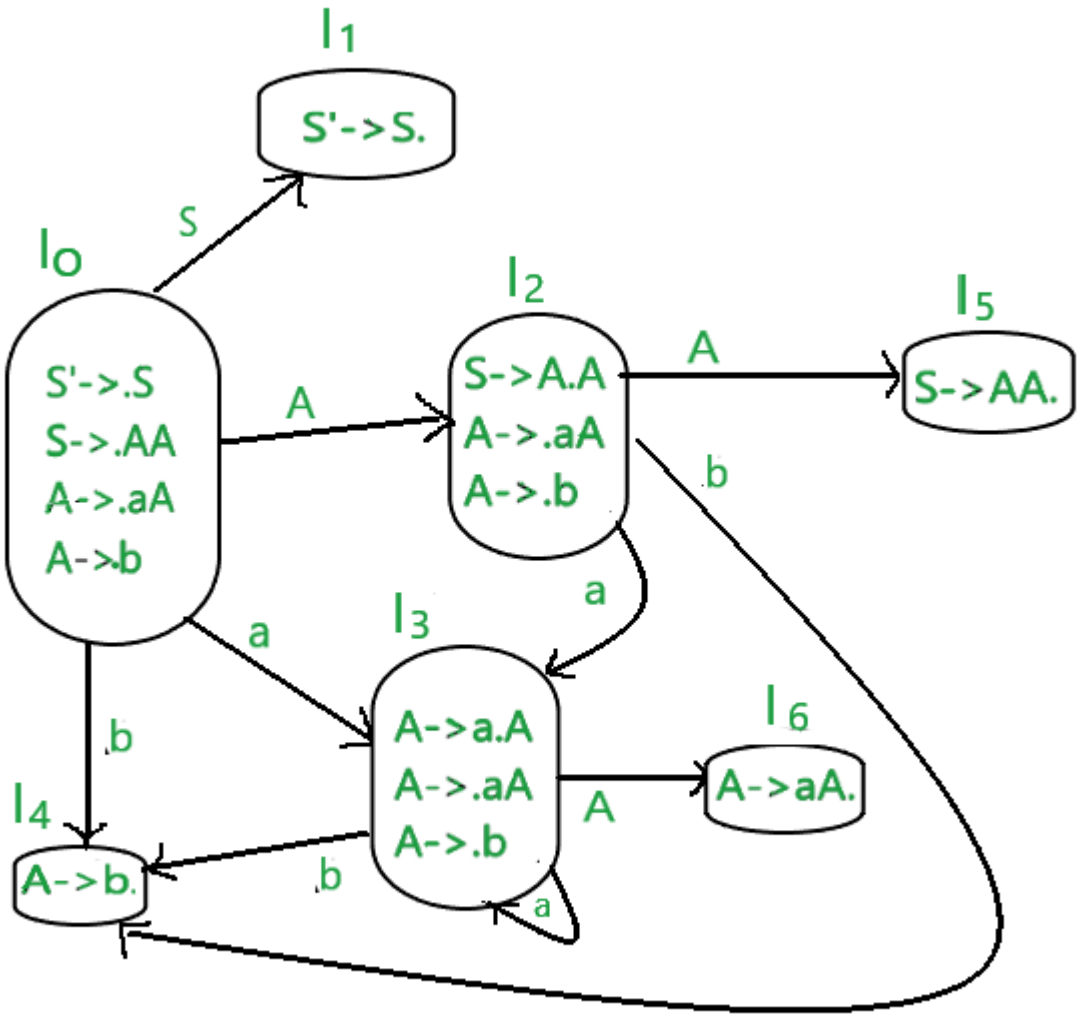
Symbols	Input	Action
	id+id$	Shift to 3
id	+id$	Reduce by T→id
T	+id$	Reduce by E→T.
E	+id$	Shift to 4
E+	id$	Shift to 3
E+id	$	Reduce by T→id
E+T	$	Reduce by E→E+T
E	$	Accept



LR(0) Parser – Example

- Construct an LR parsing table for the given context-free grammar -
- S-->AA
  A-->aA|b
- The augmented grammar of the given grammar is:-
- S'-->.S [0th production]
  S-->.AA [1st production]
  A-->.aA [2nd production]
  A-->.b [3rd production]

LR(0) Parser – Collection Items



LR(0) Parser – Parse Table

State	a	b	$	S	A
I_0	S3	S4		I_1	I_2
I_1			ACCEPT		
I_2	S3	S4			I_5
I_3	S3	S4			I_6
I_4	R(A->b)	R(A->b)	R(A->b)		
I_5	R(S->AA)	R(S->AA)	R(S->AA)		
I_6	R(A->aA)	R(A->aA)	R(A->aA)		

LR(0) Parser – Example “abaab”

Step	Stack (Before)	Input	Action	Stack (After)
1	0	abaab$	S3, push (a, 3)	0a3
2	0a3	baab$	S4, push (b, 4)	0a3b4
3	0a3b4	aab$	Reduce(A->b), Pop 2 (b, 4), Push (A, 6)	0a3A6
4	0a3A6	aab$	Reduce(A->aA), Pop 4 (a, 3, A, 6), Push (A, 2)	0A2
5	0A2	aab$	S3, push (a, 3)	0A2a3
6	0A2a3	ab$	S3, push (a, 3)	0A2a3a3
7	0A2a3a3	b$	S4, push (b, 4)	0A2a3a3b4
8	0A2a3a3b4	$	Reduce(A->b), Pop 2 (b, 4), Push (A, 6)	0A2a3a3A6
9	0A2a3a3A6	$	Reduce(A->aA), Pop 4 (a, 3, A, 6), Push (A, 6)	0A2a3A6
10	0A2a3A6	$	Reduce(A->aA), Pop 4 (a, 3, A, 6), Push (A, 5)	0A2A5
11	0A2A5	$	Reduce (S->AA), Pop 4 (A, 2, A, 5), Push(S, 1)	0S1
12	0S1	$	ACCEPT	



UNIT-V

Syntax-Directed Definition/Translation

- Syntax-Directed Definitions (SDD) and Syntax-Directed Translation (SDT) map grammar rules to semantic actions, computing values or generating code during parsing.
- An SDD is a high-level definition (using attributes/rules)
- SDT is an implementation scheme embedding actions directly into productions.

Syntax-Directed Definition (SDD)

- An SDD focuses on *what* to compute using attributes associated with grammar symbols.
- Evaluating Arithmetic Expression 4 * 5 + 6
- The parse tree is annotated bottom-up.
- F->id sets F.val = 4, 5, 6
- T multiplies 4 X 5 = 20
- E add 20 + 6 = 26

Production	Semantic Rule
E->E+T	E.Val = E.val + T.val
E->T	E.val = T.val
T->T*F	T.val = T.val*F.val
T->F	T.val = F.val
F->id	F.val=id.lexval

Syntax-Directed Translation (SDT)

- An SDT places actions (using {} braces) directly inside production rules.
- Convert an infix expression to a postfix expression

Production	Semantic Action
E->E+T	{print('+')}
E->T	
T->T*F	{print('*')}
T->F	
F->id	{print('id.lexval')}

Syntax-Directed Translation (SDT)

- Syntax-Directed Translation (SDT) uses grammar rules + semantic actions
- Associates meaning (semantics) with syntax structures
- Used in:
  - Evaluate (Expression)
  - Translation (Postfix)
  - Type checking
  - Intermediate code generation
  - Symbol table management

Syntax-Directed Translation - Examples

- Evaluating Arithmetic Expression
  - E → E + T
    E → T
    T → num
- Semantic Rules
  - E.val = E1.val + T.val
    E.val = T.val
    T.val = num.lexval
- Infix to Postfix Conversion
  - Grammar with Actions
  - E → E + T { print('+') }
    E → T
    T → num { print(num) }

Syntax-Directed Translation - Examples

- Type Checking
  - E → E1 + E2
    E → id
  - E.type = integer if E1.type = integer and E2.type = integer
    E.type = type(id)

Types of SDDs

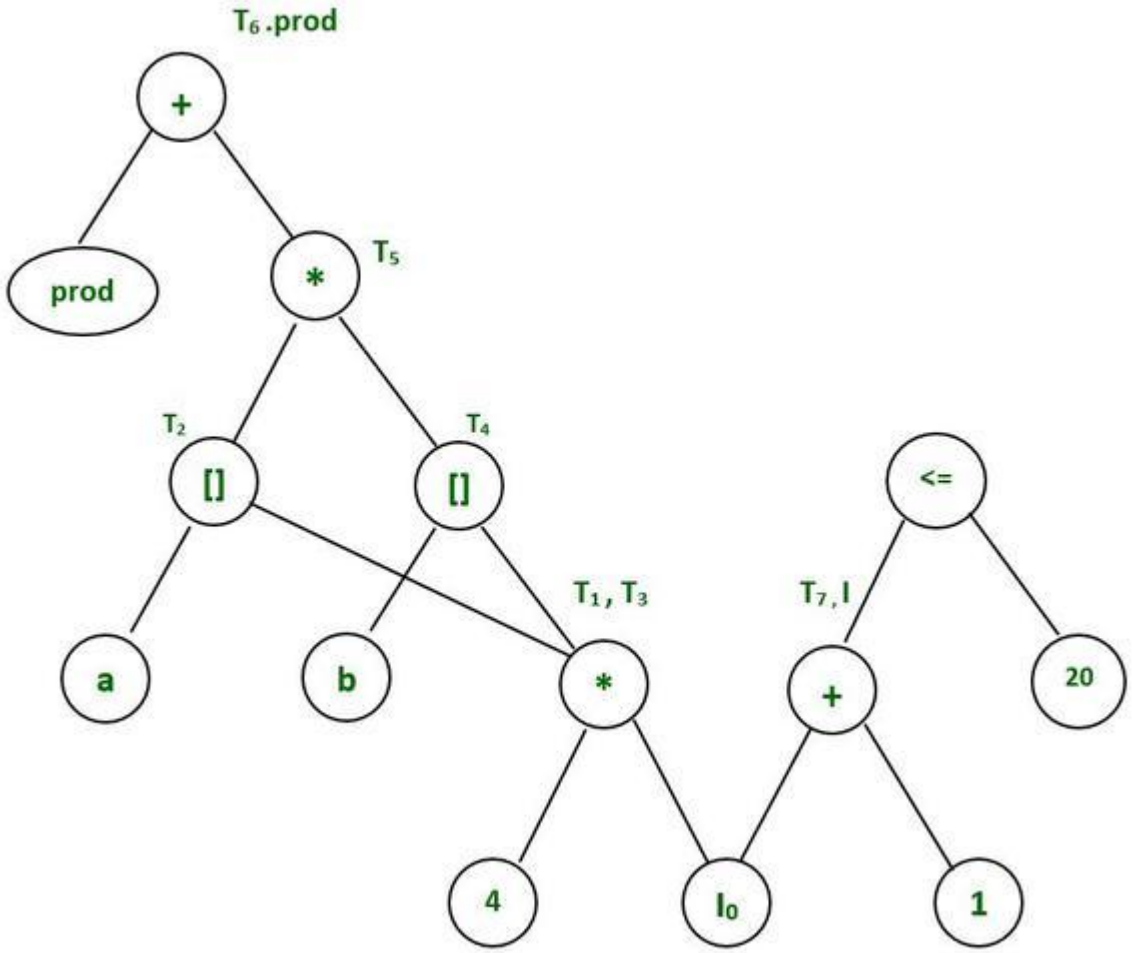
- S-Attributed SDD
  - Only synthesized attributes
  - Evaluated bottom-up (postorder traversal)
  - Used in LR parsing
- L-Attributed SDD
  - Allows inherited attributes (restricted)
  - Evaluated left-to-right
  - Used in LL parsing

Intermediate-Code Generation

- Need for Intermediate Code
  - Machine-independent representation
  - Enables optimization
  - Improves portability
- Directed Acyclic Graph (DAG)
  - Represents common subexpressions
  - DAG shares repeated nodes → avoids re-computation
- Three-Address Code (TAC)
  - Instructions with at most 3 operands
    - Example: x = y op z

DAG - Example

$T_1 := 4*I_0$
$T_2 := a[T_1]$
$T_3 := 4*I_0$
$T_4 := b[T_3]$
$T_5 := T_2 * T_4$
$T_6 := prod + T_5$
$prod := T_6$
$T_7 := I_0 + 1$
$I_0 := T_7$
if I_0 <= 20 goto 1



Intermediate-Code Generation

- Three-Address Code (TAC) Examples
  - x = a + b * c
  - t1 = b * c
  - x = a + t1
- Types of TAC Statements
  - Binary
  - Unary
  - Assignment
  - Conditional jumps
- Representations of TAC
  - Quadruples: op, arg1, arg2, result

Run-Time Environments

- Organization of memory during execution
  - Code segment
  - Stack
  - Heap
  - Static data
- Stack Allocation of Space
  - Function calls
  - Local variables
- Activation Record (Stack Frame):
  - Parameters, Return address, Local variables, etc

Heap Management

- Used for **dynamic allocation**
- Functions: malloc() and free()
- Garbage Collection

Feature	Stack	Heap
Allocation	Automatic	Manual
Speed	Fast	Slower
Lifetime	Function scope	Dynamic

Code Optimization

- Intermediate Code Generation
  - Three Address Code (TAC)
  - Quadruples
  - Triples
- Code Optimization
  - Constant folding
  - Dead code elimination
  - Common subexpression elimination
- Final Code Generation
  - Converting optimized intermediate code into target machine code.